

Архиватор на основе алгоритма Хаффмана

Долженко Александра Борисовна

Группа: 25.Б42-мм

Кафедра: *системного программирования*

Научный руководитель: *Литвинов Юрий Викторович*

Номер семестра практики: 2

1. Введение

Алгоритм Хаффмана — метод сжатия без потерь, основанный на построении оптимального префиксного кода. Применяется в JPEG, MP3, gzip. В рамках практики разработано консольное приложение для сжатия и восстановления файлов, что позволяет освоить бинарные деревья, битовые потоки и управление памятью в C.

2. Постановка задачи

Целью работы является получение практического опыта низкоуровневого программирования на языке C путём реализации консольного приложения для сжатия и восстановления файлов по алгоритму Хаффмана.

Для достижения цели требуется решить следующие задачи:

1. Выполнить обзор алгоритма Хаффмана и существующих методов сжатия без потерь.
2. Спроектировать архитектуру приложения и формат сжатого файла.
3. Реализовать сжатие и распаковку файлов.
4. Провести эксперименты по оценке степени сжатия и производительности.

3. Обзор

Алгоритм Хаффмана предложен в 1952 году [1]. Он строит префиксный код на основе частот символов: чем чаще символ, тем короче код. Другие методы сжатия без потерь: LZ77 [2], LZW [3], арифметическое кодирование [4], RLE [5]. Хаффман уступает им по степени сжатия, но проще и быстрее; в gzip [6] он используется вместе с LZ77.

Вывод: алгоритм не является лучшим по сжатию, но широко применяется на практике в сочетании с другими методами и остаётся классическим учебным примером.

4. Архитектура и технические детали

Программа состоит из четырёх модулей: **bitOperation** (битовый буфер), **queue** (очередь с приоритетом), **huffmanTree** (построение дерева и кодов), **huffman** (логика сжатия и распаковки).

Для построения дерева используется бинарная куча со сложностью $O(\log n)$ на операцию. Отдельно обрабатывается случай одного уникального символа: создаётся фиктивный родительский узел.

4.1. Алгоритм сжатия

1. **Подсчёт частот:** файл читается побайтово, для каждого из 256 символов считается частота, сохраняется размер файла.
2. **Построение дерева:** для каждого символа с ненулевой частотой создаётся лист, листья помещаются в очередь с приоритетом (min-heap). Два узла с минимальной частотой объединяются в родителя, пока в очереди не останется один корень. Для одного символа создаётся фиктивный родитель, у которого левый потомок — лист, а правый — NULL.
3. **Генерация кодов:** рекурсивный обход дерева; влево — дописывается 0, вправо — 1. Код сохраняется в листе.
4. **Запись заголовка:** маркер 0xFF, число уникальных символов, пары «символ — частота», исходный размер.
5. **Кодирование:** файл читается повторно, для каждого байта ищется код в дереве, биты накапливаются в буфере и записываются по 8 штук.

4.2. Алгоритм распаковки

1. **Чтение заголовка:** проверяется маркер, читается число символов, восстанавливается таблица частот и исходный размер.
2. **Восстановление дерева:** по частотам строится дерево тем же алгоритмом.
3. **Декодирование:** биты читаются по одному; от корня при 0 идём влево, при 1 — вправо. При достижении листа символ записывается в файл, текущий узел сбрасывается на корень.
4. **Завершение:** процесс идёт до восстановления исходного размера.

4.3. Формат сжатого файла

- **Маркер** (1 байт, 0xFF);
- **Количество уникальных символов** (4 байта);
- **Таблица частот**: пары «символ (1 байт) — частота (8 байт)»;
- **Исходный размер** (8 байт);
- **Сжатые данные**: битовый поток.

Хранятся только символы с ненулевой частотой. Последний байт может быть неполным; лишние биты игнорируются при распаковке, так как процесс завершается по исходному размеру.

4.4. Обоснование выбора представления таблицы частот

Таблица хранится в виде пар «символ (1 байт) — частота (8 байт)». У данной реализации есть альтернативы, например: полная таблица из 256 частот и канонические коды Хаффмана [7].

Преимущества: для текстов (70–90 уникальных символов, что примерно соответствует одному языку) заголовок занимает 630–810 байт вместо 2048;

Ограничения: для полного алфавита заголовок занимает 2304 байта, что больше фиксированной таблицы из 256 частот (2048 байт); при сжатии файлов менее 1 КБ они наоборот увеличиваются из-за размера заголовка.

Возможное улучшение: выбор формата через маркер (0xFF — нынешняя реализация, 0xFE — полная таблица, 0xFD — однобайтовые коды) и тд.

5. Эксперименты

5.1. Методика и степень сжатия

Использовались файлы различных типов: тексты, повторяющиеся, нулевые, смешанные данные, JPEG, бинарные файлы. Степени сжатия приведены в таблице 1.

Таблица 1: Результаты сжатия тестовых файлов

Файл	Исходный (В)	Сжатый (В)	Экономия (%)
Текстовые данные			
text_en_dict_1MB.txt	985,084	551,749	43.99
text_en_dict_100k.txt	102,400	60,417	41.00
text_en_dict_10k.txt	10,240	5,950	41.89
text_58b.txt	58	251	-332.76
Повторяющиеся данные			
repeated_10MB.txt	10,485,760	3,713,782	64.58
repeated_50k.txt	42,000	16,594	60.50
single_char_10K.txt	10,000	1,281	87.19
Нулевые данные			
zero_10MB.bin	10,485,760	1,310,742	87.50
zero_100k.bin	102,400	12,822	87.48
zero_10k.bin	10,240	1,302	87.29
Изображения			
Image_450k.JPG	454,850	451,021	0.84
Image_14MB.jpg	14,568,090	14,547,943	1.00
Бинарные файлы			
bash_binary.bin	1,446,024	1,128,089	21.99
mixed_10MB.bin	10,485,760	8,880,266	15.32
mixed_10K.bin	5,120	7,434	-45.19
random_100k.bin	102,400	104,717	-2.26
random_10k.bin	10,240	12,556	-22.61

Результаты. Эффективность сжатия зависит от энтропии данных: высокая степень — на нулевых (до 87.50%), повторяющихся (60–87%) и текстовых (41–45%) данных; умеренная — на исполняемых файлах (bash_binary.bin: 21.99%), поскольку машинный код содержит много нулей, повторяющихся инструкций и строк, но энтропия выше, а число символов приближается к 256; низкая — на уже сжатых (Image_14MB.jpg: 1.00%); увеличение размера — на случайных (random_100k.bin: -2.26%) и очень маленьких (text_58b.txt: -332.76%) файлах, так как заголовок превышает объём данных. Таким образом, представление таблицы частот парами «символ — частота» оптимально для текстов, но уступает фиксированной таблице на данных с полным алфавитом.

5.2. Конфигурация устройства

Замеры производительности проводились на следующей конфигурации виртуальной машины: процессор 12th Gen Intel Core i7-1280P (4 ядра доступно), оперативная память 7.8 ГБ, накопитель VBOX HARDDISK 50 ГБ, операционная система Ubuntu 24.04.3 LTS, ядро Linux 7.0.0-31-generic, файловая система ext4, компилятор GCC 13.3.0, флаги компиляции -O2, CMake 3.28.3, среда VirtualBox.

5.3. Производительность

Результаты замеров времени приведены в таблице 2.

Таблица 2: Время сжатия файлов ($N = 10$ прогонов).

Файл	Размер (B)	Время, сек
Текстовые данные		
text_en_dict_1MB.txt	985,084	0.128 ± 0.006
text_en_dict_100k.txt	102,400	0.017 ± 0.002
text_en_dict_10k.txt	10,240	0.007 ± 0.001
text_58b.txt	58	$0.005 \pm < 0.001$
Повторяющиеся данные		
repeated_10MB.txt	10,485,760	0.316 ± 0.015
single_char_10K.txt	10,000	0.006 ± 0.001
Нулевые данные		
zero_10MB.bin	10,485,760	0.159 ± 0.007
zero_100k.bin	102,400	$0.007 \pm < 0.001$
zero_10k.bin	10,240	0.005 ± 0.001
Смешанные данные		
mixed_10MB.bin	10,485,760	4.801 ± 0.213
Изображения		
Image_14MB.jpg	14,568,090	6.145 ± 0.124
Image_450k.JPG	454,850	0.201 ± 0.005
Бинарные файлы		
bash_binary.bin	1,446,024	0.549 ± 0.011
random_100k.bin	102,400	0.052 ± 0.002
random_10k.bin	10,240	0.011 ± 0.002

Выводы: время сжатия растёт с размером файла и его энтропией: наибольшая скорость — на нулевых (10 MB за 0.16 сек) и повторяющихся (10 MB за 0.32 сек) данных, наименьшая — на случайных и больших JPEG (10 MB за 6–7 сек) из-за глубокого дерева.

5.4. Проверка памяти

Valgrind не обнаружил утечек памяти: ERROR SUMMARY: 0 errors; All heap blocks were freed.

6. Заключение

В ходе работы реализован архиватор на основе алгоритма Хаффмана (сжатие и распаковка, поддержка любых типов данных, обработка краевых случаев). Эксперименты показали: наилучшее сжатие — на данных с низкой энтропией (до 87.5%), текст сжимается на 41–44%, уже сжатые данные не сжимаются. Скорость составляет от 1.3 MB/сек (случайные данные) до 50 MB/сек (нулевые), утечек памяти не обнаружено.

Реализацию можно найти по адресу: https://github.com/DolzhenkoAlexa/Project_huffman (дата обращения: 24 сентября 2026 г.).

Список литературы

[1] Huffman coding. URL: https://en.wikipedia.org/wiki/Huffman_coding (дата обращения: 24 сентября 2026 г.).

[2] LZ77 and LZ78. URL: https://en.wikipedia.org/wiki/LZ77_and_LZ78 (дата обращения: 24 сентября 2026 г.).

[3] Lempel-Ziv-Welch. URL: <https://en.wikipedia.org/wiki/Lempel-Ziv-Welch> (дата обращения: 24 сентября 2026 г.).

[4] Arithmetic coding. URL: https://en.wikipedia.org/wiki/Arithmetic_coding (дата обращения: 24 сентября 2026 г.).

[5] Run-length encoding. URL: https://en.wikipedia.org/wiki/Run-length_encoding (дата обращения: 24 сентября 2026 г.).

[6] GNU Gzip. URL: <https://www.gnu.org/software/gzip/> (дата обращения: 24 сентября 2026 г.).

[7] Canonical Huffman Code. URL: https://en.wikipedia.org/wiki/Canonical_Huffman_code (дата обращения: 24 сентября 2026 г.).